Supabase SQL

Supabase SQL

```
-- per-day sent/opened count (local timezone onujayi) — 7-day trend chart er
jonno
create or replace function public.email_daily(
  p_license_key text,
  p_days int,
  p_offset_min int
)
returns json
language sql
security definer
as $$
  with days as (
    select generate_series(
      ((now() - (p_offset_min || ' minutes')::interval)::date - (p_days - 1)),
      ( now() - (p_offset_min || ' minutes')::interval)::date,
      interval '1 day'
    )::date as d
  ),
  agg as (
    select ((sent_at - (p_offset_min || ' minutes')::interval))::date as d,
           count(*) filter (where status = 'sent')  as sent,
           count(*) filter (where opened = true)     as opened
    from public.email_events
    where license_key = p_license_key
      and sent_at >= (now() - (p_offset_min || ' minutes')::interval) -
((p_days) || ' days')::interval
    group by 1
  )
  select coalesce(json_agg(json_build_object(
    'day',    to_char(days.d, 'YYYY-MM-DD'),
    'sent',   coalesce(agg.sent, 0),
    'opened', coalesce(agg.opened, 0)
  ) order by days.d), '[]'::json)
  from days left join agg on agg.d = days.d;
$$;
```

sql

```sql
-- ═══════════════════════════════════════════════════════════════
-- LeadOutfy — Date-wise tracking buckets (Phase 6)
-- Returns today / yesterday / last 7 days / last 30 days / all-time
-- counts in ONE call. Day boundaries use Bangladesh time (Asia/Dhaka).
-- Additive: does NOT change the existing email_stats function.
-- ═══════════════════════════════════════════════════════════════
create or replace function public.email_stats_buckets(p_license_key text)
returns json
language sql
security definer
as $$
  with e as (
    select status, opened, replied, sent_at,
           (sent_at at time zone 'Asia/Dhaka')::date as d
    from public.email_events
    where license_key = p_license_key
  ),
  cur as ( select (now() at time zone 'Asia/Dhaka')::date as today )
  select json_build_object(
    'today', (
      select json_build_object(
        'total',  count(*),
        'sent',   count(*) filter (where status = 'sent'),
        'failed', count(*) filter (where status = 'failed'),
        'opened', count(*) filter (where opened),
        'replied',count(*) filter (where replied))
      from e, cur where e.d = cur.today),
    'yesterday', (
      select json_build_object(
        'total',  count(*),
        'sent',   count(*) filter (where status = 'sent'),
        'failed', count(*) filter (where status = 'failed'),
        'opened', count(*) filter (where opened),
        'replied',count(*) filter (where replied))
      from e, cur where e.d = cur.today - 1),
    'week', (
      select json_build_object(
        'total',  count(*),
        'sent',   count(*) filter (where status = 'sent'),
        'failed', count(*) filter (where status = 'failed'),
        'opened', count(*) filter (where opened),
        'replied',count(*) filter (where replied))
```

```
      from e where sent_at >= now() - interval '7 days'),
    'month', (
      select json_build_object(
        'total',  count(*),
        'sent',   count(*) filter (where status = 'sent'),
        'failed', count(*) filter (where status = 'failed'),
        'opened', count(*) filter (where opened),
        'replied',count(*) filter (where replied))
      from e where sent_at >= now() - interval '30 days'),
    'all', (
      select json_build_object(
        'total',  count(*),
        'sent',   count(*) filter (where status = 'sent'),
        'failed', count(*) filter (where status = 'failed'),
        'opened', count(*) filter (where opened),
        'replied',count(*) filter (where replied))
      from e)
  );
$$;
```

Tab 3

```sql
-- track_open ke update: send-er 12 sec-er moddhe open hole machine prefetch
dhore baad
create or replace function public.track_open(p_tracking_id text)
returns void
language plpgsql
security definer
as $$
begin
  update public.email_events
     set opened          = true,
         open_count      = open_count + 1,
         first_opened_at  = coalesce(first_opened_at, now()),
         last_opened_at   = now()
   where tracking_id = p_tracking_id
     and now() - sent_at > interval '12 seconds';   -- prefetch filter
end;
$$;
```

Tab 4

```sql
-- ═══════════════════════════════════════════════════════
-- LeadOutfy — Email Tracking (Phase 1)
-- Notun table + functions. Purono kichu change kore na.
-- ═══════════════════════════════════════════════════════

create table if not exists public.email_events (
  id              uuid primary key default gen_random_uuid(),
  tracking_id     text unique not null,
  license_key     text not null,
  recipient       text,
  channel         text,                       -- gmail | custom | brevo
  subject         text,
  status          text default 'sent',        -- sent | failed
  error           text,
  opened          boolean default false,
  open_count      int default 0,
  replied         boolean default false,
  sent_at         timestamptz default now(),
  first_opened_at timestamptz,
  last_opened_at  timestamptz,
  replied_at      timestamptz
);

create index if not exists email_events_license_idx  on public.email_events
(license_key);
create index if not exists email_events_tracking_idx on public.email_events
(tracking_id);

-- RLS on (backend service key eta bypass kore, anon key dhukte parbe na)
alter table public.email_events enable row level security;

-- ── Function 1: pixel load hole open mark kore ──
create or replace function public.track_open(p_tracking_id text)
returns void
language plpgsql
security definer
as $$
begin
  update public.email_events
     set opened          = true,
         open_count      = open_count + 1,
         first_opened_at = coalesce(first_opened_at, now()),
```

```
        last_opened_at  = now()
   where tracking_id = p_tracking_id;
end;
$$;

-- ── Function 2: ekjon customer er sob count (stats) ──
create or replace function public.email_stats(p_license_key text)
returns json
language sql
security definer
as $$
  select json_build_object(
    'total',   count(*),
    'sent',    count(*) filter (where status = 'sent'),
    'failed',  count(*) filter (where status = 'failed'),
    'opened',  count(*) filter (where opened = true),
    'replied', count(*) filter (where replied = true)
  )
  from public.email_events
  where license_key = p_license_key;
$$;
```

important

```
SELECT prosrc
FROM pg_proc
WHERE proname = 'deduct_credit';
```

imported

```sql
SELECT routine_name, routine_definition
FROM information_schema.routines
WHERE routine_name = 'deduct_credit';
```